Finding structure without labels: customer segments and unusual orders

How do we group customers, or spot odd cases, when nobody gave us the answers?

AI and Data Science Professional | Session 18 of 24 | Module D: Machine learning

KLAi Professional Learning | Jotham Martin Wambi

Today in one view: what you will leave with

By the end of today you can



Build recency, frequency and monetary value (RFM) variables, then apply a log transform and standardisation

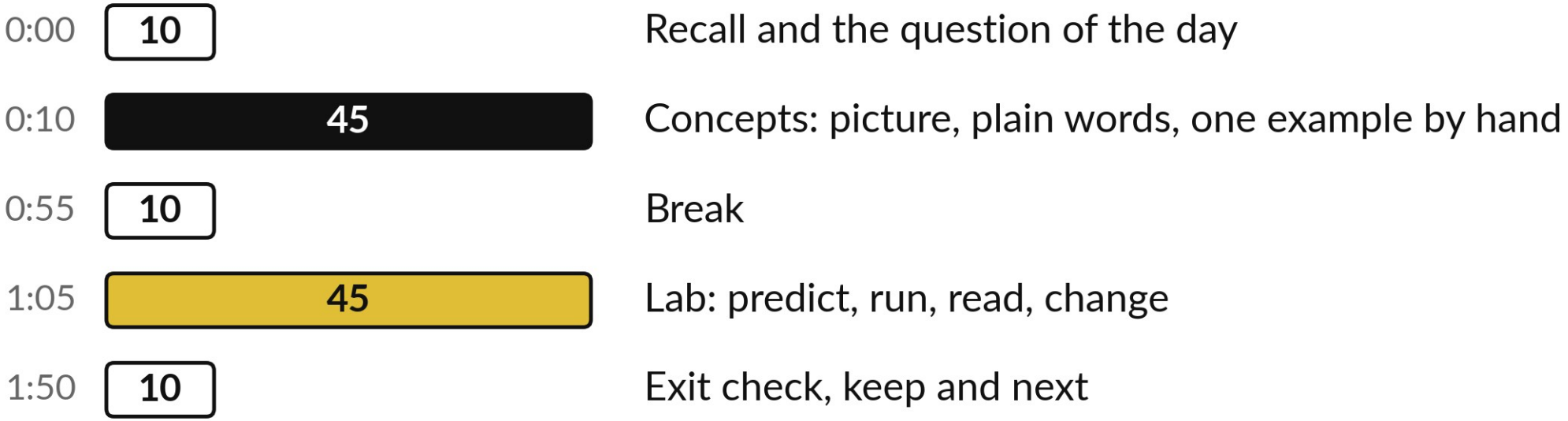


Run k-means clustering, choose k with the elbow method, reduce three variables to two with PCA, and name each segment in business words



Flag unusual orders and customers with a percentile rule and an isolation forest, and say what to do with them

Your two hours: concepts first, then a Python lab



Ideas with pictures first. Then you open the notebook. Then the checks and what you keep.

Three questions from last time

1. A model scores very well on its training rows and much worse on its test rows. What is that called?

2. A forest reports a high permutation importance for one variable. What can you safely say?

3. Two models score about the same. Which one do you ship, and what is the rule called?



Answer aloud first. Then we check together.

Answers to the three questions from Session 17

	Question	Short answer
	**A model scores very well on its training rows and much worse on its test rows. What is that called?**	Overfitting. The distance between the two scores is the generalisation gap.
	**A forest reports a high permutation importance for one variable. What can you safely say?**	The model relies on that variable. It does not show a cause.
	**Two models score about the same. Which one do you ship, and what is the rule called?**	The simpler one. The rule is parsimony.

Words we use today

	Word	What it means
	**Unsupervised learning (for example clustering, anomaly detection)**	Learning with no answer column. The algorithm looks for groups or odd rows by itself.
	**RFM analysis**	Three numbers per customer: recency (days since the last order), frequency (number of orders) and monetary value (total spend).
	**Feature scaling and log transform**	Standardisation (z-score) puts variables on one scale. A log transform squeezes a long right tail.
	**k-means clustering**	Splits records into k clusters around k centroids. Inertia measures how tight they are. The elbow method suggests k.
	**Principal component analysis (PCA)**	A dimensionality reduction method: it mixes many variables into a few principal components that keep most of the variance.

Supervised learning has an answer column. Unsupervised learning has none

Sessions 14 to 17: supervised

The loans table has a default column.

The model learns to predict it.

We score it against the known answer.

Today: unsupervised

The orders have no segment column.

Nobody told us the groups.

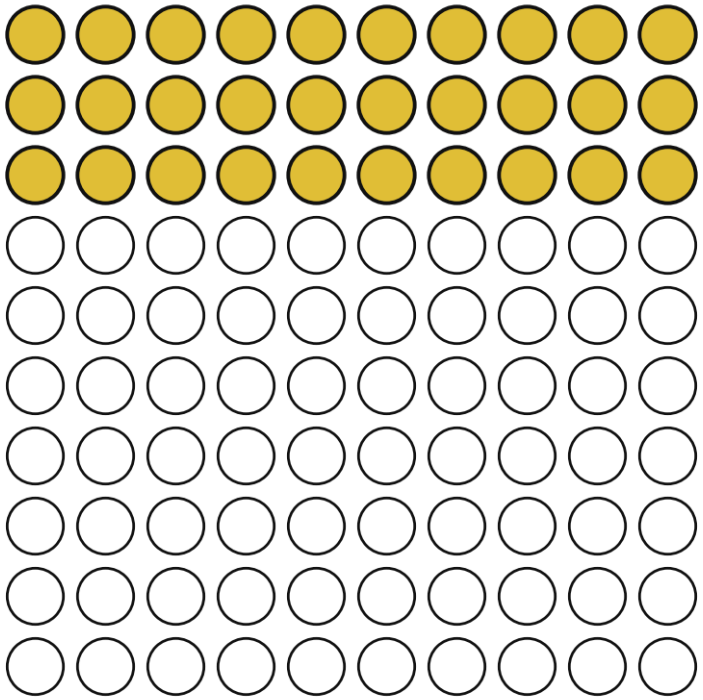
We look for structure, then judge it ourselves.

No accuracy to chase. Inertia and the silhouette score help. A person judges.

Walk-in orders have no customer_id, so 30 percent of orders cannot join a customer segment

30%

9,171 of 30,229 orders are walk-ins. We keep the 21,058 loyalty orders, from 2,499 customers.



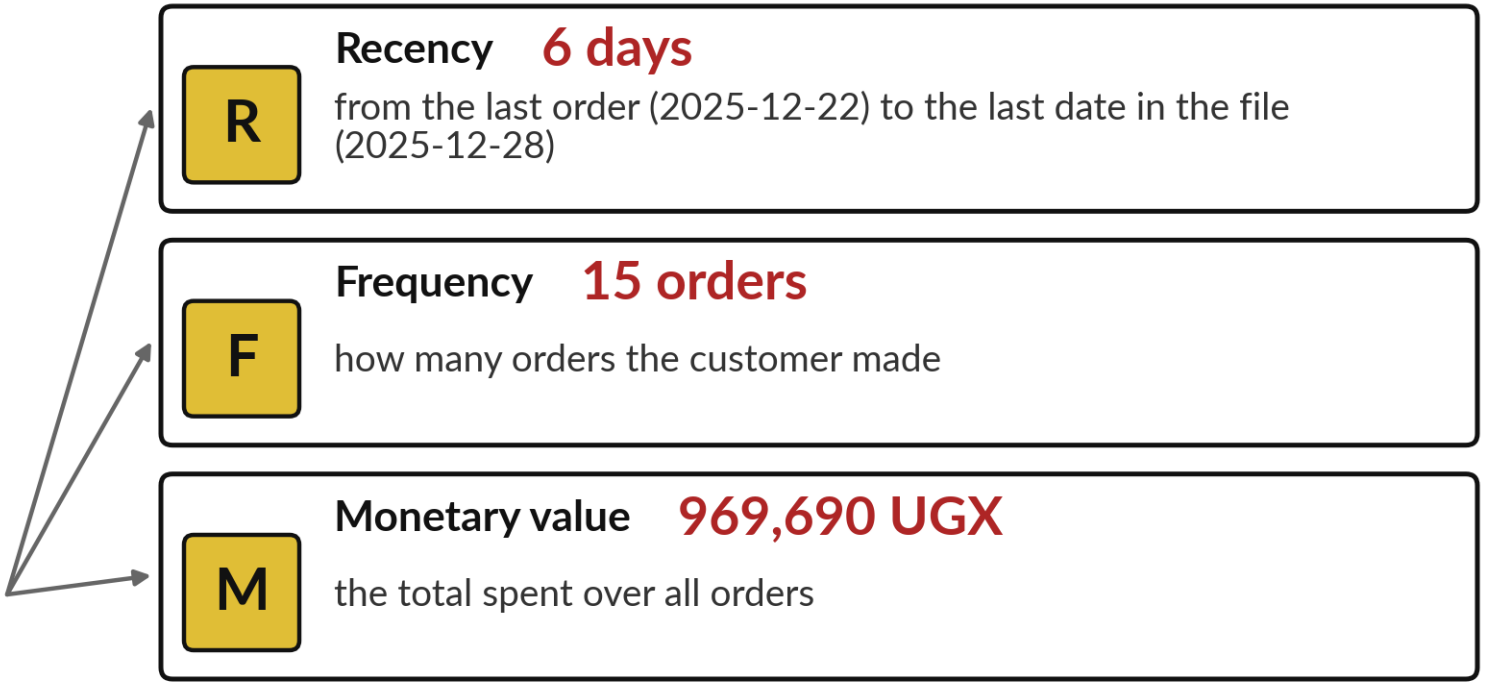
orders in every 100 are walk-ins

RFM analysis turns one customer's orders into three numbers: recency, frequency and monetary value

Orders of customer C0001

order_date	order_value_ugx
2025-09-17	43,600
2025-11-02	385,600
2025-11-12	3,500
2025-11-27	32,780
2025-12-22	37,000

(the last 5 of 15 orders)



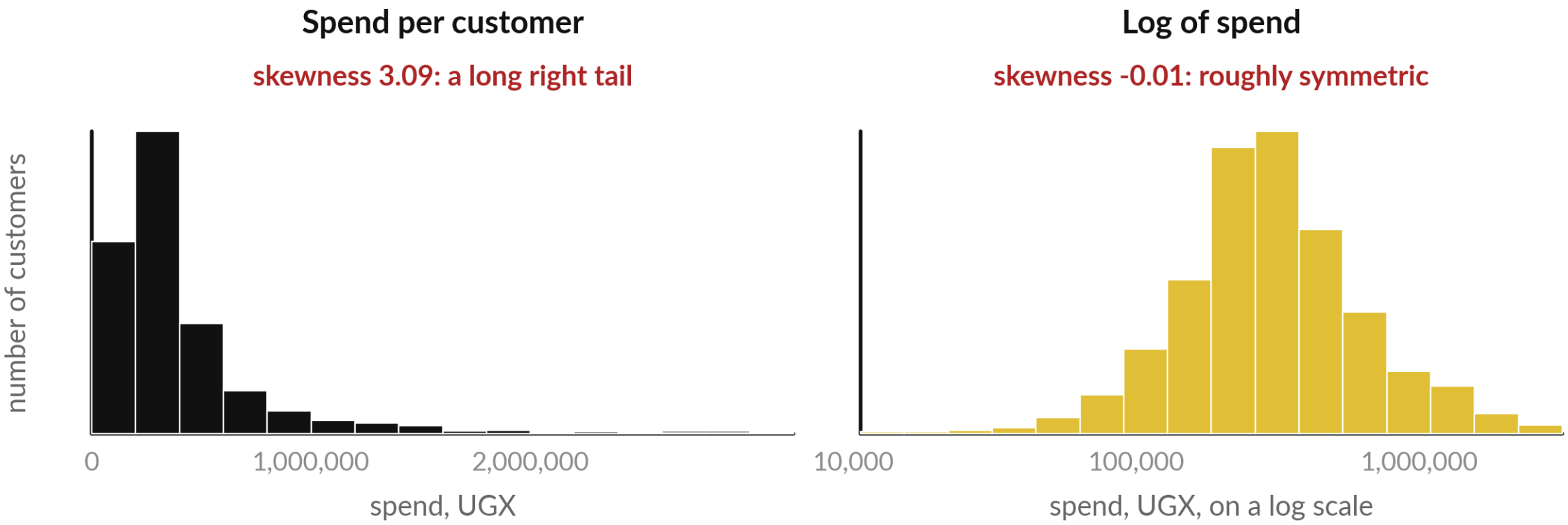
One record per customer: C0001 made 15 orders, the last on 2025-12-22.

The RFM table has one record per customer: 2,499 records and three variables

Customer	Recency (days)	Frequency (orders)	Monetary value (UGX)
C0001	6	15	969,690
C0002	119	4	107,745
C0003	1	6	437,465
Mean	78.5	8.4	399,687
Median	46	8	290,090
Standard deviation	88.7	2.9	374,078

Each record is one loyalty customer. The mean spend is above the median: a few large spenders pull it up.

A log transform squeezes the long right tail: the skewness of spend falls from 3.09 to -0.01



Skewness measures lopsidedness. We log all three variables: frequency goes from 0.37 to -0.65.

Standardisation turns each variable into a z-score: how many standard deviations from the mean

z-score

$$z = \frac{x - \bar{x}}{s}$$

Customer C0001, spend

$$z = \frac{13.78 - 12.61}{0.75} = 1.57$$

Symbol	What it means	In our data
x	The value of one customer: here the log of spend	13.78 for C0001 (spend 969,690 UGX)
x-bar and s	The mean and the standard deviation of that variable over all customers	12.61 and 0.75
z	The z-score: 0 is average, +1 is one standard deviation above	1.57: well above average

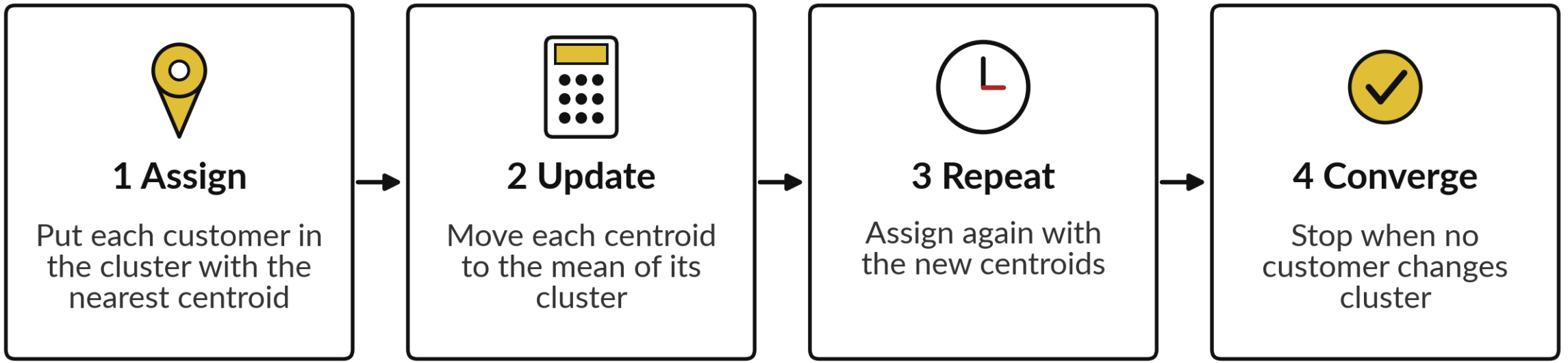
Computed from unrounded values, so a hand calculation differs by 0.01. In scikit-learn: StandardScaler.

Feature scaling puts recency, frequency and spend on one scale before k-means measures distance

	Variable	Standard deviation, raw	Standard deviation, logged and scaled
	**Recency**	88.7 days	1.0
	**Frequency**	2.9 orders	1.0
	**Monetary value**	374,078 UGX	1.0

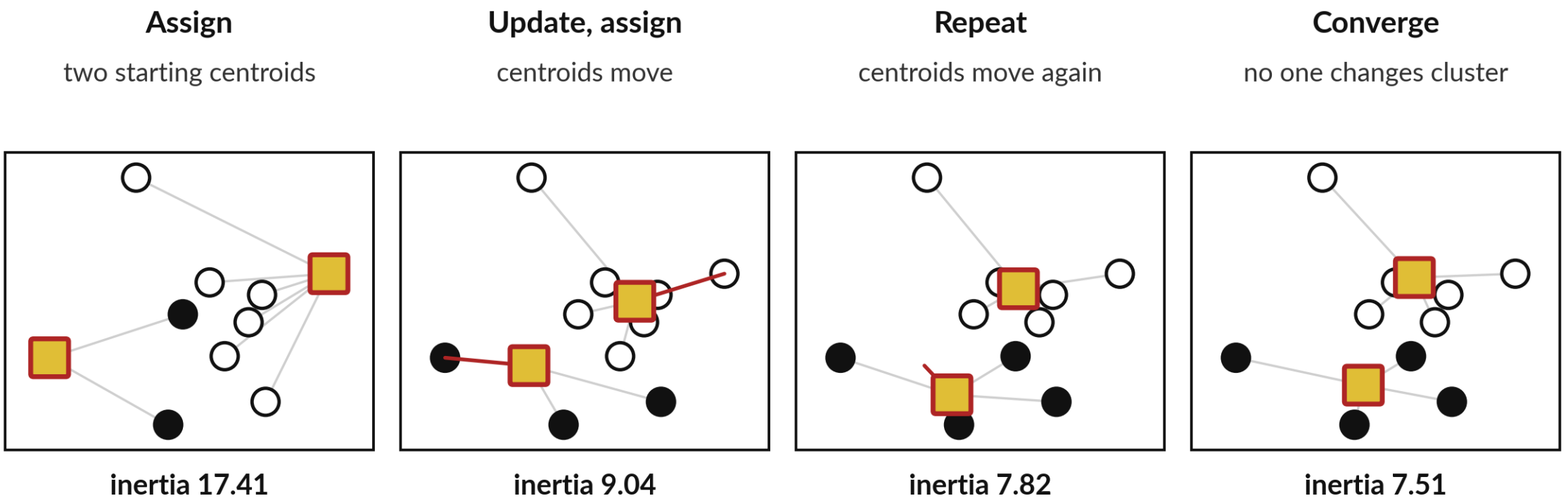
k-means uses distance. Without scaling, spend (in shillings) would decide almost every distance and the other two would count for nothing.

k-means clustering repeats two moves until the groups stop changing



k is the number of clusters you choose. A centroid is the mean point of a cluster. Inertia is the sum of squared distances from each customer to its centroid.

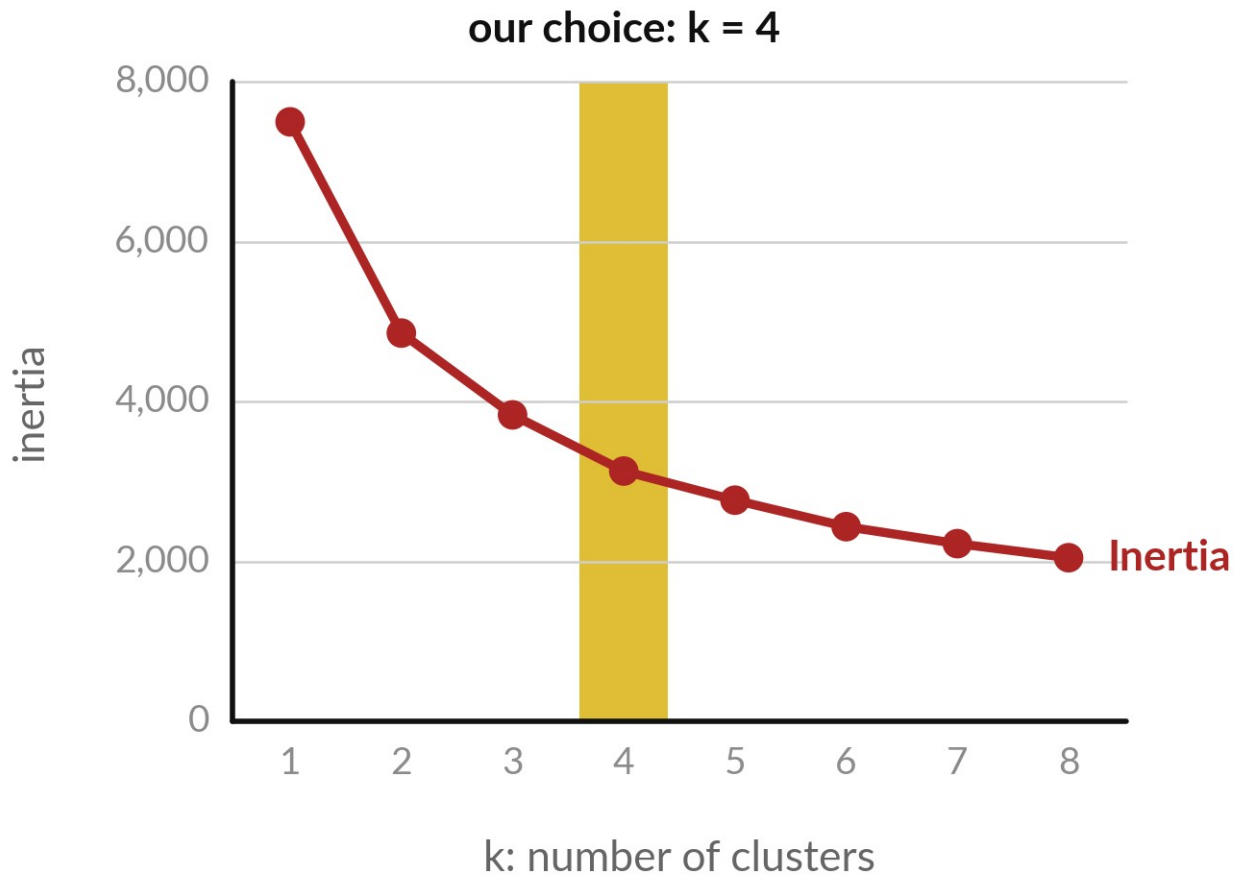
k-means on ten real customers: the centroids move and the inertia falls until nothing changes



x: scaled log recency. y: scaled log spend. Gold square: centroid. Filled and hollow dots: the two clusters.

k = 2. Inertia falls from 17.41 to 7.51 in 3 updates.

The elbow method plots inertia against k and looks for the bend



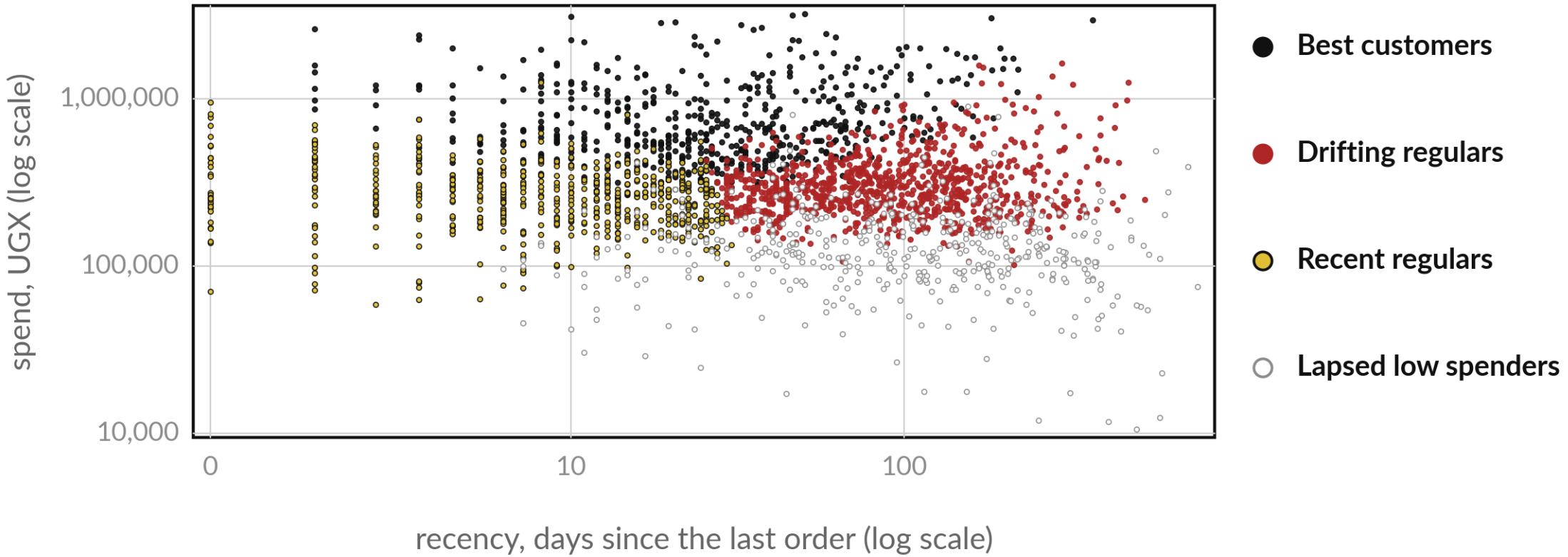
Inertia always falls as k grows. Look for where the fall slows.

k = 3: 3,832. k = 4: 3,130. k = 5: 2,765.

The bend is gentle, so 3, 4 or 5 are all defensible.

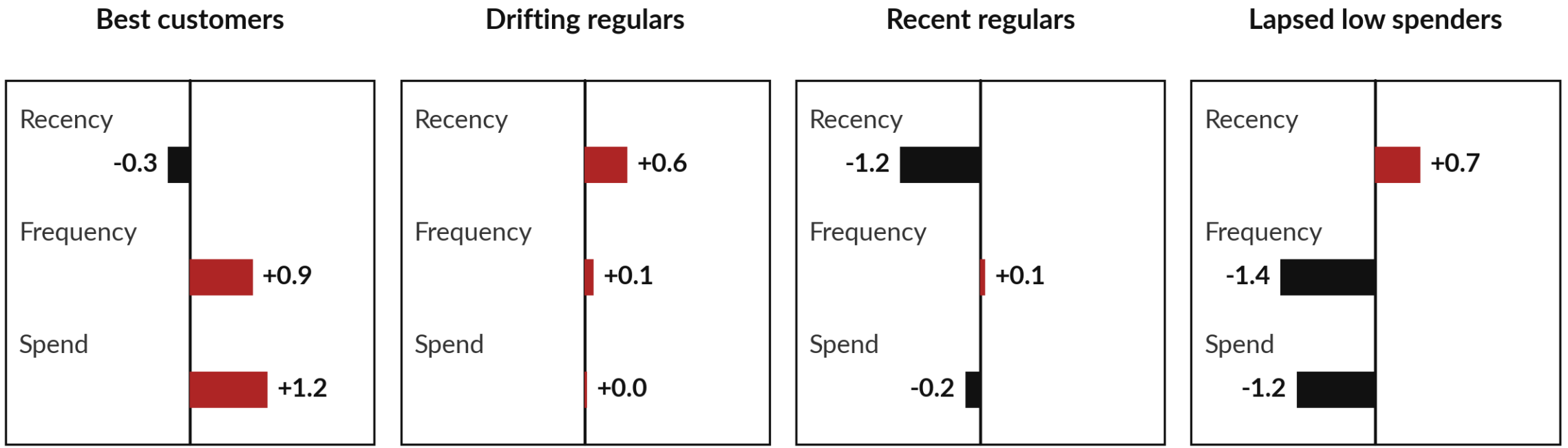
We pick 4: four segments can each be named and acted on.

Four customer segments on recency and spend: the groups touch, so the edges are soft



One dot per customer (2,499). Colours are k-means segments; your run differs by up to 5 percent.

What each segment is like: how far its average sits from the average customer



Distance of the segment average from the average customer, in standard deviations (0 = average)

Bars: standard deviations from the average customer. Red above, black below.

Naming each segment in business words: who they are, in real units

	Segment name	Customers	Recency (days)	Orders	Spend (UGX)	Share of spend
	**Best customers**	576	40	11.5	833,080	48%
	**Drifting regulars**	898	112	8.5	330,332	30%
	**Recent regulars**	560	11	8.3	277,601	16%
	**Lapsed low spenders**	465	143	4.7	143,805	7%

Averages per customer; your own run differs by up to about 5 percent. Share of spend is out of 998,818,340 UGX.

A segment is only useful if each name comes with an action

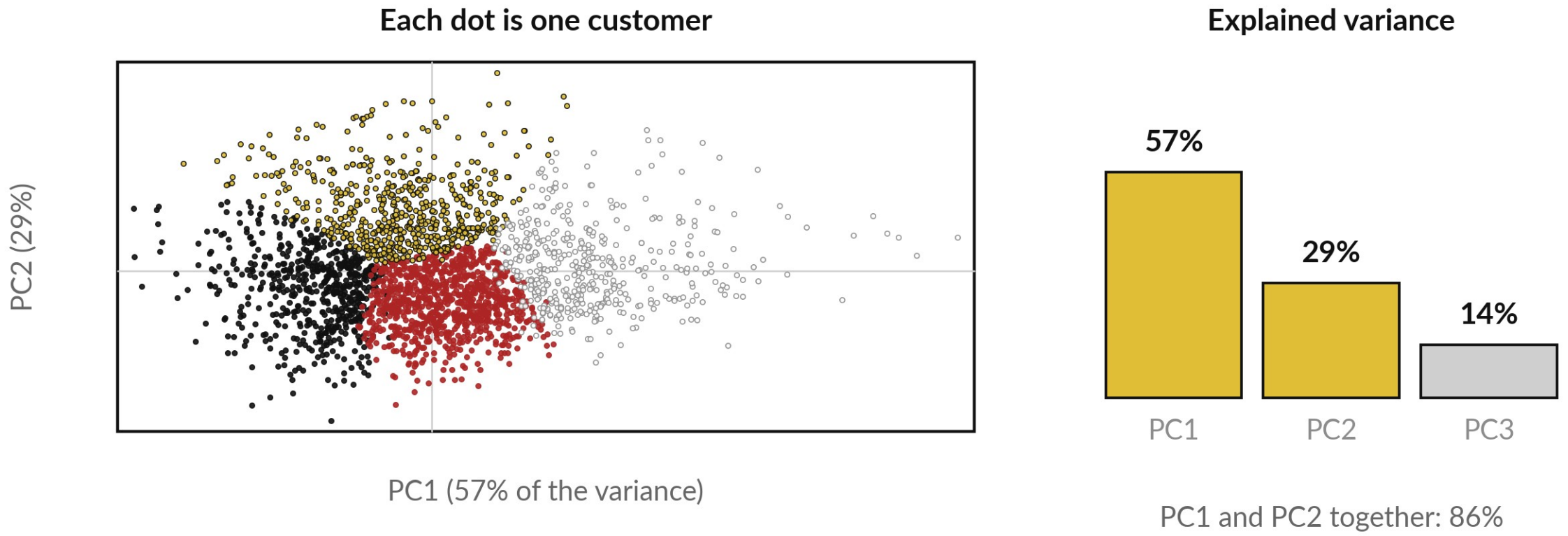
	Segment name	Who they are	What we would do
	**Best customers**	576 customers; last order 40 days ago; 11.5 orders	Protect them: early access to promotions and one named contact.
	**Drifting regulars**	898 customers; last order 112 days ago; 8.5 orders	Win them back before they lapse: a reminder offer in the next month.
	**Recent regulars**	560 customers; last order 11 days ago; 8.3 orders	Grow the basket: promote higher-value products to them.
	**Lapsed low spenders**	465 customers; last order 143 days ago; 4.7 orders	Send a low-cost reactivation message and do not spend heavily.

Re-running k-means moves segment sizes by up to 67 customers: do not quote a size to the last customer

Segment, by size	Smallest over 60 starts	Largest over 60 starts
Smallest	433	473
Second smallest	545	576
Second largest	571	638
Largest	870	902

60 runs with different random starts. The inertia stays between 3,129 and 3,130.

PCA squeezes three variables into two principal components that keep about 86 percent of the variance



Dimensionality reduction: fewer variables, same customers. Colours are the k-means segments.

Each principal component is a weighted mix of recency, frequency and spend

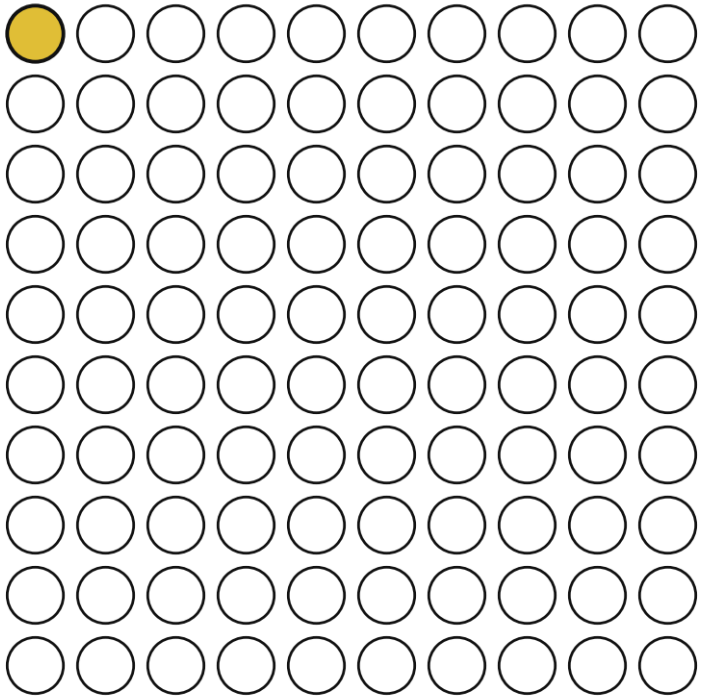
	Variable	PC1 weight	PC2 weight	PC3 weight
	**Recency**	+0.41	-0.90	+0.16
	**Frequency**	-0.66	-0.17	+0.73
	**Monetary value**	-0.63	-0.40	-0.67
	**Explained variance**	57.4%	29.1%	13.5%

The sign of a component is arbitrary: flipping every sign gives the same picture. Components are listed from most to least variance.

Anomaly detection, rule one: flag orders above the 99th percentile (the percentile rule)

1%

303 of 30,229 orders are above the 99th percentile, 468,620 UGX. Median of flagged: 829,200 UGX. Median of all orders: 23,200 UGX.



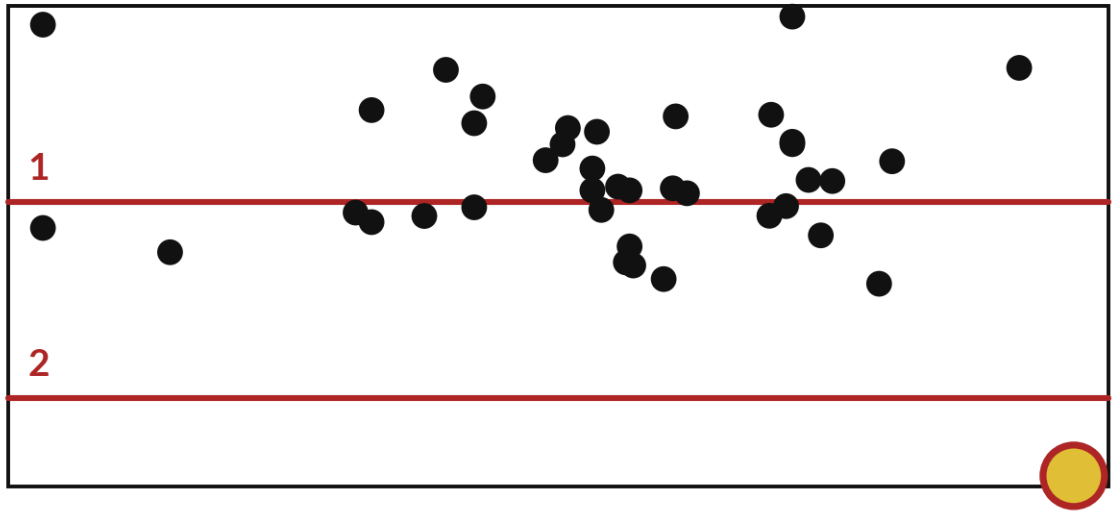
orders in every 100 are flagged

An isolation forest isolates an odd record with few random cuts, and a typical record needs many

An odd customer

recency high, spend very low

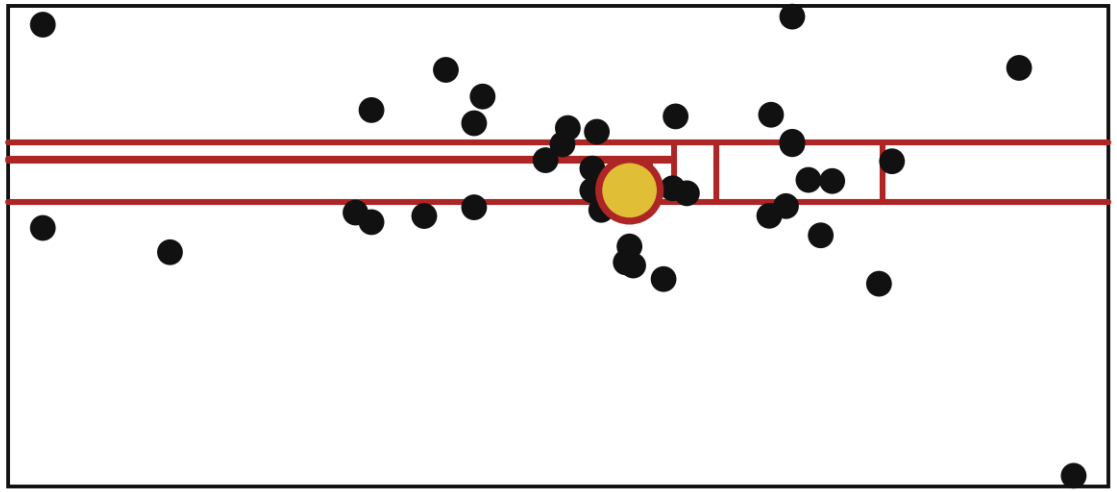
isolated after 2 cuts in this run



A typical customer

near the middle of the cloud

isolated after 10 cuts in this run



Mean of 300 runs: 2.7 and 9.8 cuts. The real forest uses 100 trees on 256 customers each.

An isolation forest flags 25 unusual customers: lapsed low spenders and extreme spenders

	Customer	Recency (days)	Orders	Spend (UGX)
🔒	**C0605**	551	1	12,390
🔒	**C0913**	559	1	22,880
🔒	**C0379**	471	2	10,600
🔍	**Median of the 25 flagged**	152	3	41,930
👥	**Median of all customers**	46	8	290,090

Of the 25 flagged: 12 are long-lapsed with low spend, 7 spent over 800,000 UGX, 6 are other. Our NumPy isolation forest: scikit-learn will differ a little.

An anomaly is a record to review, and the review decides what happens

Flagging is the start of the work.



Check for an error: a typing slip, a duplicate or a test order.



Check for a real and important case: a wholesale buyer or a fraud.



Record the decision and the reason.

Where you will use this at work: group records, then act on the groups

	Your job	The records you meet	What today lets you do
	**Marketing analyst at a supermarket chain, using pandas and scikit-learn**	Till orders with a loyalty id.	Build RFM, run k-means, name four segments and attach an action to each.
	**Manager of a mobile money agent network**	Transactions per agent per month.	Segment agents by recency, frequency and value, and find the agents who are drifting away.
	**Member services officer at a SACCO, using Excel**	Savings deposits and balances per member.	Export the segment table as CSV for Power BI or Excel, with each segment named and sized.
	**Internal auditor or finance controller**	Payments and expense claims.	Flag the top one percent with the percentile rule, and an isolation forest for the odd combinations.

Read the recipe before the lab: log, scale, cluster, profile

```
X = StandardScaler().fit_transform(np.log1p(rfm))
kmeans = KMeans(n_clusters=4, n_init=10,
                random_state=42)
rfm["segment"] = kmeans.fit_predict(X)
print(kmeans.inertia_)
rfm.groupby("segment")[["recency",
    "frequency", "spend"]].mean().round(0)
```

In plain words

Line 1 takes the log of each variable, then scales it to a z-score.

Line 2 sets k = 4, 10 random starts, and fixes the random state.

Line 3 fits k-means and stores each customer's segment number.

Line 4 prints the inertia.

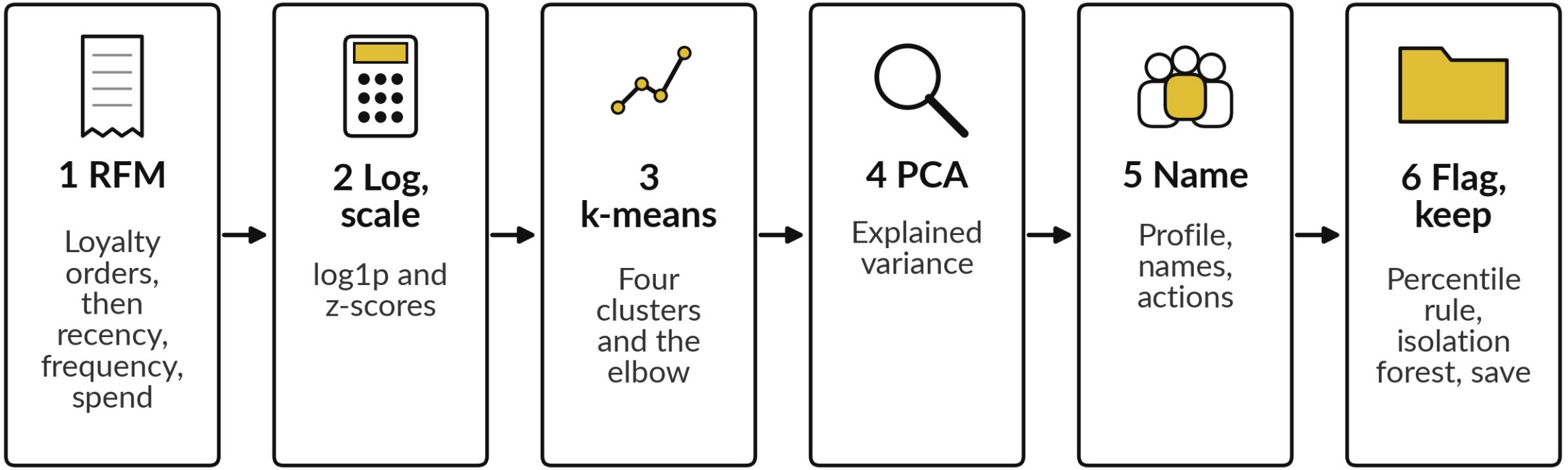
Line 5 profiles each segment in real units.



Part 2: the Python lab

Close the slides. Open the notebook.

The lab has six parts, and you keep a segment file with names and actions



Six parts, eight tasks in 45 minutes. In each task, predict first, run second, read what the output tells you, then change one thing.

Check your numbers before you move on

What you compute	You should get
Loyalty orders, and customers	21,058 and 2,499
RFM of customer C0001	6 days, 15 orders, 969,690 UGX
Skewness of spend, raw and logged	3.09 and -0.01
Inertia for k = 1, and for k = 4	7,497, and about 3,130 (within 5)
Segment sizes, k = 4, smallest to largest	about 433-473, 545-576, 571-638, 870-902
Explained variance of PC1, PC2, PC3	0.574, 0.291, 0.135
99th percentile of order value, and orders above it	468,620 UGX, 303 orders
Customers flagged by IsolationForest (contamination 0.01)	about 25

k-means uses random starts, so segment sizes vary a little between runs. The notebook checks allow for that.

Five likely mistakes in the lab and how to fix them

	What you see	What it means	What to do
✗	**30,229 customers instead of 2,499**	The walk-in orders (blank customer_id) are still in.	Drop the blank customer_id values first, with dropna on customer_id.
✗	**Recency is negative or huge**	The end date came from today's date.	Use the last order_date in the file as the end date.
✓	**Segment 0 is different from the slide**	The segment numbers are only labels.	Match segments by their averages.
✗	**Spend dominates the clusters**	The variables were not scaled.	Apply log1p and StandardScaler before KMeans.
🧮	**Inertia for k = 1 is not 7,497**	The data were not standardised, or a row was dropped.	Three standardised variables over 2,499 customers give 7,497.

Exit check: three questions, then compare with the short answers

	Question	Short answer
	**Why do we take the log of spend and then standardise before k-means?**	The log squeezes the long tail (skewness 3.09 to -0.01). Standardising gives the three variables one scale, so none dominates the distance.
	**The elbow chart bends gently between k = 3 and k = 5. How do you choose k?**	Treat the elbow as a hint. Choose the k whose segments you can name and act on, and say it is a judgement.
	**An isolation forest flags 25 customers. What do you do next?**	Review each one: an error, or a real and important case. Record the decision. Never delete them automatically.

Save your notebook and customer_segments.csv

What you keep from today

Keep your customer segment table: each of the 2,499 loyalty customers with its RFM numbers, its segment name and one action, saved as a file, plus a list of unusual orders to review.

Our question today

How do we group customers, or spot odd cases, when nobody gave us the answers?

Next session

Session 19: Forecasting and a first end-to-end model. Project milestone 3.

Take away

Your lab file

keep/customer_segments.csv

The names

4 segments, 4 actions

Next session

Forecasting